Manual 4 · Actividad 2

GESTIÓN DE ESQUEMAS E ÍNDICES EN AZURE SQL DATABASE

Alumno	**Rafael Max Cajahuaringa Camasca**
Código	t04119c
Institución	Universidad Peruana Los Andes (UPLA)
Curso	Base de Datos II
Material de origen	Actividad 2 · Guías Semana 4
Base de datos	Grados_Y_Titulos
Plataforma	Azure SQL Database (PaaS)

1. Presentación y propósito

Este manual desarrolla cinco prácticas sobre esquemas lógicos, índices clúster, índices no clúster con columnas incluidas, índices compuestos y análisis de fragmentación. Amplía el material de laboratorio con pasos de verificación, interpretación, precauciones y criterios de evaluación. Las capturas son las evidencias del archivo original.

2. Requisitos previos

• Acceso al editor de consultas de Azure SQL Database y a la base Grados_Y_Titulos.

• Permisos suficientes para consultar catálogos del sistema; permisos DDL solo para las actividades de creación autorizadas.

• Comprobar que existan las tablas y columnas requeridas antes de crear índices.

• Realizar cambios únicamente en una base de pruebas o con autorización del responsable.

3. Conceptos clave

Concepto	Descripción
Esquema	Espacio lógico de nombres para agrupar objetos y gestionar permisos.
Índice clúster	Estructura de filas ordenadas por clave; una por tabla.
Índice no clúster	Estructura de búsqueda independiente de la organización principal de filas.
INCLUDE	Columnas adicionales disponibles en el nivel hoja del índice.
Índice compuesto	Índice cuya clave contiene dos o más columnas.
Fragmentación	Desorden lógico de páginas o baja densidad que puede afectar el rendimiento.

4. Guía N.º 01 · Consolidación y verificación de esquemas lógicos

Objetivo

Verificar la existencia de los esquemas Catalogo, Academico, Tramite, Evaluacion, Documento y Seguridad.

Fundamento y aplicación

Los esquemas separan los objetos por módulo funcional y permiten administrar autorizaciones con mayor claridad. El principio de mínimo privilegio requiere conceder únicamente los permisos necesarios para cada rol.

Procedimiento de laboratorio

• Seleccionar la base Grados_Y_Titulos.

• Ejecutar la creación condicional de los seis esquemas si se cuenta con autorización.

• Consultar sys.schemas y registrar los seis nombres visibles.

Consulta T-SQL para la práctica

```
IF NOT EXISTS (SELECT 1 FROM sys.schemas WHERE name='Catalogo') EXEC('CREATE SCHEMA Catalogo');
IF NOT EXISTS (SELECT 1 FROM sys.schemas WHERE name='Academico') EXEC('CREATE SCHEMA Academico');
IF NOT EXISTS (SELECT 1 FROM sys.schemas WHERE name='Tramite') EXEC('CREATE SCHEMA Tramite');
IF NOT EXISTS (SELECT 1 FROM sys.schemas WHERE name='Evaluacion') EXEC('CREATE SCHEMA Evaluacion');
IF NOT EXISTS (SELECT 1 FROM sys.schemas WHERE name='Documento') EXEC('CREATE SCHEMA Documento');
IF NOT EXISTS (SELECT 1 FROM sys.schemas WHERE name='Seguridad') EXEC('CREATE SCHEMA Seguridad');
SELECT schema_id, name AS NombreEsquema FROM sys.schemas
WHERE name IN ('Catalogo','Academico','Tramite','Evaluacion','Documento','Seguridad');
```

Interpretación y comprobación

Se espera una fila por cada esquema existente. La ausencia de un nombre exige revisar permisos, base seleccionada y errores de ejecución.

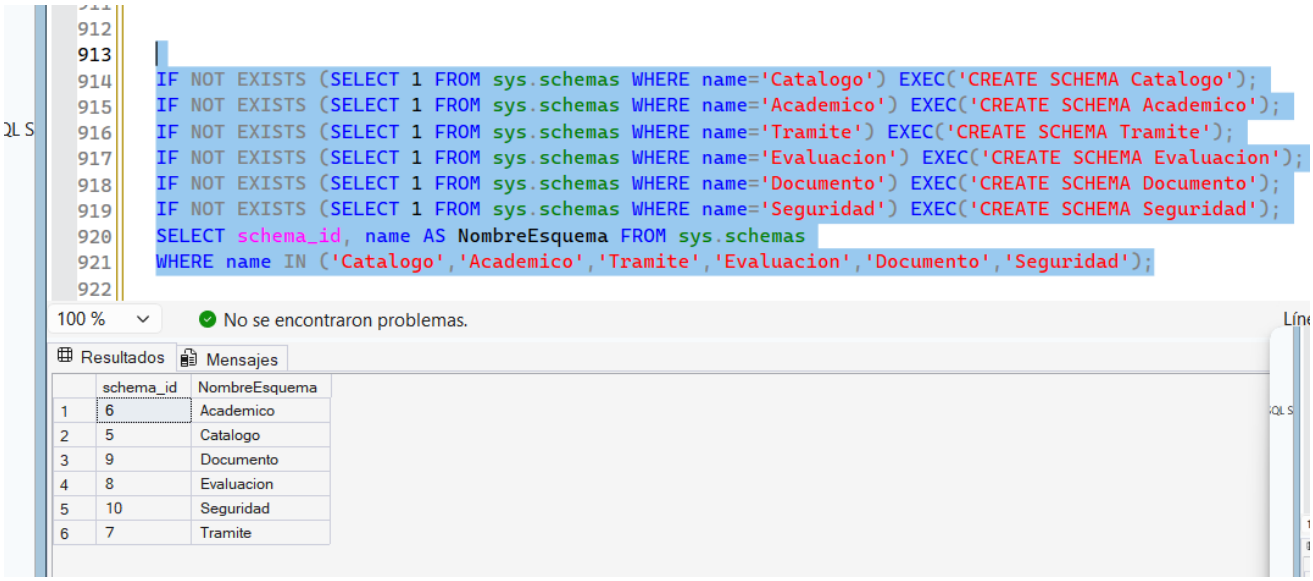


Figura 1. Evidencia proporcionada en la guía original; no representa una ejecución nueva.

Pregunta de validación

¿Cómo mejora la separación por esquemas el mínimo privilegio en RBAC?

Respuesta: Permite otorgar acceso a objetos de un módulo concreto sin conceder acceso indiscriminado a otros módulos.

Ampliación técnica

Crear esquemas no asigna permisos automáticamente. Un diseño RBAC adecuado exige roles específicos, GRANT selectivos y revisión periódica de autorizaciones.

5. Guía N.º 02 · Auditoría de índices clúster estratégicos

Objetivo

Identificar los índices clúster de las tablas y valorar la selección de sus claves.

Fundamento y aplicación

Un índice clúster organiza las filas por la clave de índice. Solo puede existir uno por tabla. Una clave estable, estrecha y, cuando convenga, secuencial puede reducir divisiones de páginas, aunque el diseño depende de los patrones de consulta.

Procedimiento de laboratorio

• Ejecutar la auditoría sobre sys.indexes.

• Revisar esquema, tabla, nombre y tipo de índice.

• Registrar tablas que no aparecen y analizar si son heaps o si requieren otro diseño.

Consulta T-SQL para la práctica

```
SELECT s.name AS Esquema, t.name AS Tabla,
       i.name AS IndiceClustered, i.type_desc AS Tipo
FROM sys.indexes AS i
JOIN sys.tables AS t ON i.object_id=t.object_id
JOIN sys.schemas AS s ON t.schema_id=s.schema_id
WHERE i.type=1
ORDER BY s.name,t.name;
```

Interpretación y comprobación

Cada fila identifica un índice CLUSTERED existente. La consulta no demuestra por sí sola que el índice sea óptimo; se necesita conocer la carga y el plan de consultas.

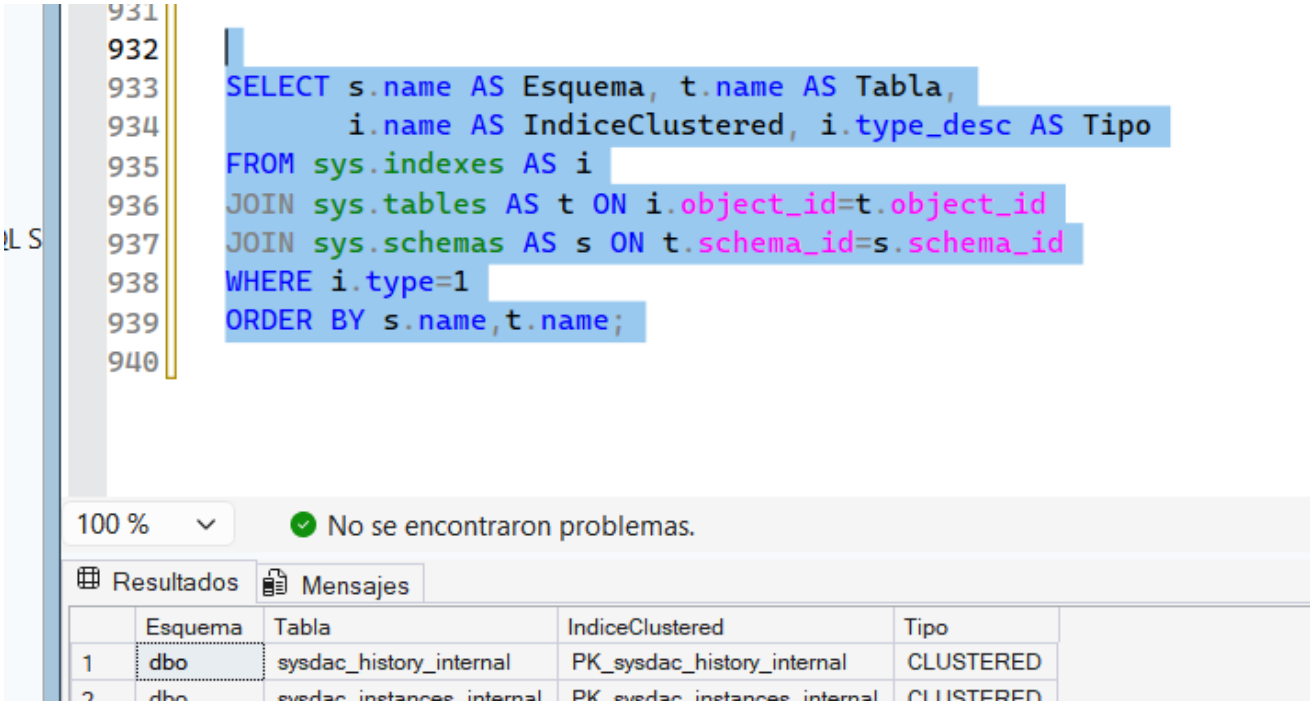


Figura 2. Evidencia proporcionada en la guía original; no representa una ejecución nueva.

Pregunta de validación

¿Por qué solo puede haber un índice clúster por tabla y qué aporta una clave secuencial?

Respuesta: Porque las filas solo pueden tener una organización clúster; una clave secuencial puede disminuir inserciones intermedias y divisiones de páginas.

Ampliación técnica

No se recomienda agregar un índice clúster sin revisar claves primarias, índices actuales y patrones de acceso. Crear índices nuevos consume espacio y aumenta el costo de escrituras.

6. Guía N.º 03 · Índices no clúster con INCLUDE

Objetivo

Crear o verificar índices cubiertos para consultas frecuentes de Academico.Persona.

Fundamento y aplicación

INCLUDE incorpora columnas no clave al nivel hoja de un índice no clúster. Si una consulta obtiene del índice todas las columnas que necesita, puede evitar búsquedas adicionales a la tabla base.

Procedimiento de laboratorio

• Verificar que Academico.Persona y las columnas del script existan.

• Crear los índices solo si están ausentes y existe autorización.

• Comprobar sus nombres en sys.indexes y probar una consulta real.

Consulta T-SQL para la práctica

```
IF OBJECT_ID('Academico.Persona','U') IS NOT NULL
AND NOT EXISTS (SELECT 1 FROM sys.indexes
 WHERE object_id=OBJECT_ID('Academico.Persona')
```

```
 AND name='IX_Persona_DNI_Cover')
BEGIN
 CREATE NONCLUSTERED INDEX IX_Persona_DNI_Cover
 ON Academico.Persona(DNI)
 INCLUDE(Nombres,ApellidoPaterno,ApellidoMaterno,Correo,TipoPersona);
END;
SELECT name,type_desc FROM sys.indexes
WHERE object_id=OBJECT_ID('Academico.Persona');
```

Interpretación y comprobación

El resultado de sys.indexes debe mostrar el índice creado si la tabla y sus columnas existen y la operación fue autorizada. El índice adicional del material, IX_Persona_TipoPersona, puede incorporarse con una comprobación análoga.

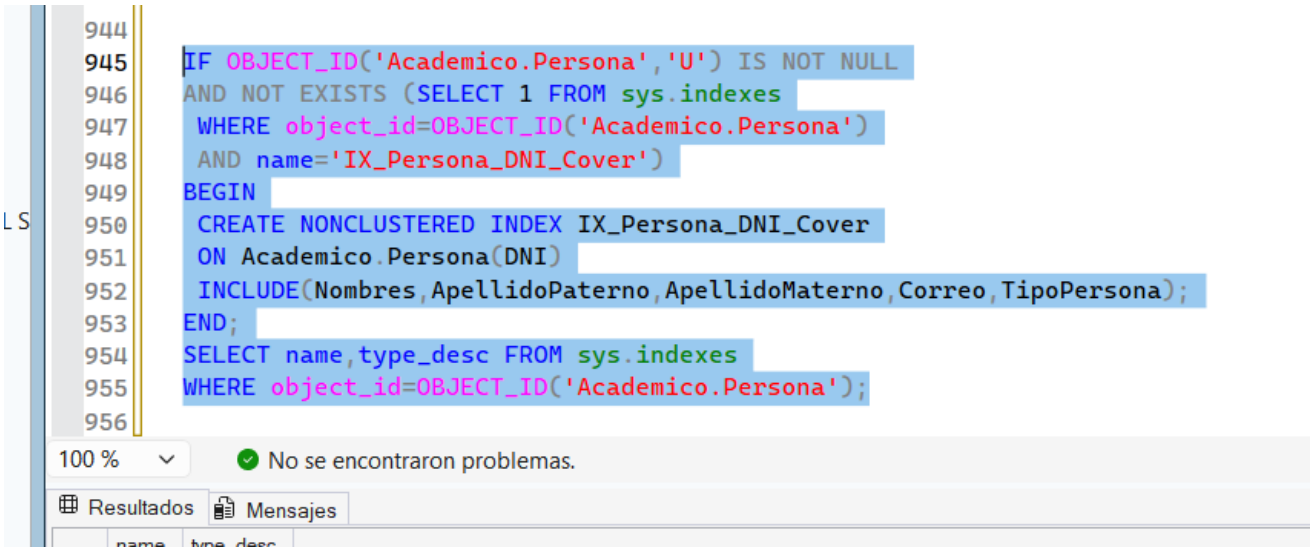


Figura 3. Evidencia proporcionada en la guía original; no representa una ejecución nueva.

Pregunta de validación

¿De qué manera INCLUDE puede evitar Key Lookups?

Respuesta: Permite resolver determinadas consultas desde el propio índice al incluir columnas proyectadas, sin ir a buscar esos datos a la estructura base.

Ampliación técnica

La cobertura depende de la consulta concreta. Demasiadas columnas INCLUDE aumentan almacenamiento y mantenimiento; conviene medir lecturas y planes antes y después.

7. Guía N.º 04 · Índices compuestos para trámites

Objetivo

Verificar la creación de índices compuestos para consultas de trámites y pagos.

Fundamento y aplicación

En un índice compuesto importa el orden de las columnas clave. En consultas con igualdad y rango suele ser útil colocar primero la columna de igualdad y después la de rango, pero el mejor diseño depende de selectividad y uso real.

Procedimiento de laboratorio

• Confirmar la existencia de Tramite.Tramite y las columnas necesarias.

• Ejecutar el índice IX_Tramite_Estado_Fecha cuando proceda.

• Auditar los índices de Tramite.Tramite y Tramite.Pago.

Consulta T-SQL para la práctica

```
IF OBJECT_ID('Tramite.Tramite','U') IS NOT NULL
AND NOT EXISTS (SELECT 1 FROM sys.indexes
 WHERE object_id=OBJECT_ID('Tramite.Tramite')
 AND name='IX_Tramite_Estado_Fecha')
BEGIN
 CREATE NONCLUSTERED INDEX IX_Tramite_Estado_Fecha
 ON Tramite.Tramite(IdEstadoTramite,FechaInicio)
 INCLUDE(IdMatricula,IdTipoTramite,IdCoordinacion);
END;
SELECT OBJECT_NAME(object_id) AS Tabla,name AS Indice
FROM sys.indexes
WHERE object_id IN (OBJECT_ID('Tramite.Tramite'),OBJECT_ID('Tramite.Pago'))
AND name IS NOT NULL;
```

Interpretación y comprobación

El índice debe figurar en la auditoría si la tabla y columnas existen. El material también propone IX_Pago_Tramite_Fecha en Tramite.Pago; comprobar su compatibilidad antes de aplicarlo.

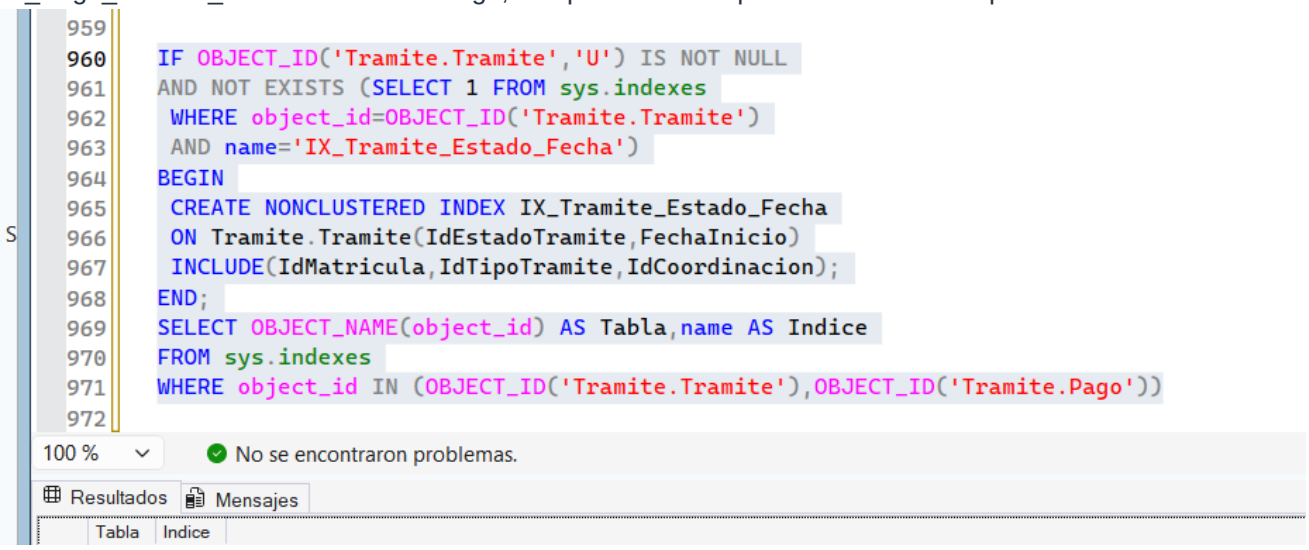


Figura 4. Evidencia proporcionada en la guía original; no representa una ejecución nueva.

Pregunta de validación

¿Por qué importa el orden de columnas en IX_Tramite_Estado_Fecha?

Respuesta: Porque la clave del índice se organiza en ese orden y condiciona qué filtros y rangos pueden aprovechar búsquedas eficientes.

Ampliación técnica

La elección no debe basarse solo en la regla igualdad-rango: también influyen ORDER BY, joins, cardinalidad, frecuencia de actualizaciones y planes de ejecución.

8. Guía N.º 05 · Auditoría y fragmentación de índices

Objetivo

Examinar índices y sus métricas de fragmentación para sustentar decisiones de mantenimiento.

Fundamento y aplicación

La DMV sys.dm_db_index_physical_stats informa fragmentación y número de páginas. El material usa umbrales orientativos: menos de 5 %, 5–30 % y más de 30 %. Estos valores no sustituyen la evaluación del tamaño y rendimiento.

Procedimiento de laboratorio

• Auditar los índices existentes.

• Consultar fragmentación mediante la DMV en modo LIMITED.

• Priorizar índices con suficientes páginas y evidencia de impacto antes de plantear mantenimiento.

Consulta T-SQL para la práctica

```
SELECT s.name AS Esquema,t.name AS Tabla,
 i.name AS NombreIndice,i.type_desc AS TipoIndice
FROM sys.indexes i
JOIN sys.tables t ON i.object_id=t.object_id
JOIN sys.schemas s ON t.schema_id=s.schema_id
WHERE i.name IS NOT NULL;

SELECT OBJECT_SCHEMA_NAME(ps.object_id) AS Esquema,
 OBJECT_NAME(ps.object_id) AS Tabla,
 i.name AS Indice,
 ps.avg_fragmentation_in_percent AS Fragmentacion,
 ps.page_count AS Paginas
FROM sys.dm_db_index_physical_stats(DB_ID(),NULL,NULL,NULL,'LIMITED') ps
JOIN sys.indexes i ON ps.object_id=i.object_id AND ps.index_id=i.index_id
WHERE ps.avg_fragmentation_in_percent > 10
ORDER BY ps.avg_fragmentation_in_percent DESC;
```

Interpretación y comprobación

La consulta puede devolver cero filas si ningún índice supera el filtro de 10 %. Una fragmentación elevada en un índice pequeño no justifica necesariamente REBUILD.

```sql
SELECT s.name AS Esquema,t.name AS Tabla,
 i.name AS NombreIndice,i.type_desc AS TipoIndice
FROM sys.indexes i
JOIN sys.tables t ON i.object_id=t.object_id
JOIN sys.schemas s ON t.schema_id=s.schema_id
WHERE i.name IS NOT NULL;

SELECT OBJECT_SCHEMA_NAME(ps.object_id) AS Esquema,
 OBJECT_NAME(ps.object_id) AS Tabla,
 i.name AS Indice,
 ps.avg_fragmentation_in_percent AS Fragmentacion,
 ps.page_count AS Paginas
FROM sys.dm_db_index_physical_stats(DB_ID(),NULL,NULL,NULL,'LIMITED') ps
JOIN sys.indexes i ON ps.object_id=i.object_id AND ps.index_id=i.index_id
WHERE ps.avg_fragmentation_in_percent > 10
ORDER BY ps.avg_fragmentation_in_percent DESC;
```

100 % No se encontraron problemas.

Resultados Mensajes

	Esquema	Tabla	NombreIndice	TipoIndice
1	dbo	sysdac_instances_internal	PK_sysdac_instances_internal	CLUSTERED
2	dbo	sysdac_instances_internal	UQ_sysdac_instances_internal	NONCLUSTERED
3	dbo	sysdac_history_internal	PK_sysdac_history_internal	CLUSTERED
4	dbo	sysdac_history_internal	UQ_sysdac_history_internal	NONCLUSTERED
5	dbo	sysdac_history_internal	IX_sysdac_history_internal	NONCLUSTERED

Figura 5. Evidencia proporcionada en la guía original; no representa una ejecución nueva.

Pregunta de validación

¿Qué criterios se consideran para REORGANIZE o REBUILD?

Respuesta: El material propone 5–30 % para REORGANIZE y más de 30 % para REBUILD; en una decisión real también deben revisarse páginas, densidad, carga, disponibilidad y costo.

Ampliación técnica

REORGANIZE y REBUILD son operaciones de mantenimiento con impacto potencial en recursos y bloqueos. No ejecutarlas automáticamente sobre producción sin evaluación y autorización.

9. Matriz de evidencias y verificación

Guía	Evidencia que debe comprobarse
01	Seis esquemas funcionales listados por sys.schemas.
02	Listado de índices CLUSTERED por tabla.
03	Existencia del índice no clúster de Persona.
04	Existencia de los índices de Tramite y Pago.
05	Fragmentación, cantidad de páginas y criterio de mantenimiento.

10. Problemas frecuentes y soluciones

Problema	Acción sugerida
Invalid object name	Comprobar base, esquema y tabla.
Invalid column name	Verificar columnas reales antes de ejecutar CREATE INDEX.
Permisos insuficientes	Solicitar autorización y permisos adecuados.
Índice ya existe	Usar la validación IF NOT EXISTS.
Sin filas en DMV	Revisar filtro de fragmentación y objetos presentes.

11. Recomendaciones

• Guardar capturas donde se lean el código y los resultados.

• Distinguir resultados reales de evidencias originales.

• Evaluar índices con planes de ejecución y consultas representativas.

• Evitar mantenimiento automático basado solo en un porcentaje.

• Documentar fecha, base, usuario técnico y versión del motor sin divulgar credenciales.

12. Conclusiones

Las cinco guías permiten ordenar objetos por dominio funcional, verificar índices existentes, comprender índices cubiertos y compuestos y observar la fragmentación. El valor del laboratorio no está solo en ejecutar T-SQL, sino en justificar técnicamente cada decisión de seguridad y rendimiento. Las consultas agregadas facilitan repetir y documentar las pruebas en Azure SQL Database.

Nota metodológica: las preguntas, temas y capturas provienen de ACTIVIDAD_2_GUIAS_SEMANA_4.pdf. Las ampliaciones, advertencias y pasos de comprobación se añadieron para mejorar el manual. No se afirma haber ejecutado las consultas en la base del estudiante.

ANEXO TÉCNICO · Solución de la Guía N.º 04

Diagnóstico y corrección de resultados vacíos en Azure SQL Database

1. Situación identificada

Al consultar los índices compuestos de Tramite.Tramite y Tramite.Pago, la consulta se ejecutó sin errores, pero devolvió una tabla vacía. La verificación con OBJECT_ID confirmó que ambas tablas figuraban como «NO EXISTE» en la base de datos conectada. Por tanto, la condición IF de creación de índices no podía generar los índices solicitados.

2. Verificación inicial

```
SELECT DB_NAME() AS BaseDatosActual;
SELECT 'Tramite.Tramite' AS Tabla,
       CASE WHEN OBJECT_ID('Tramite.Tramite','U') IS NULL
            THEN 'NO EXISTE' ELSE 'EXISTE' END AS Estado
UNION ALL
SELECT 'Tramite.Pago',
       CASE WHEN OBJECT_ID('Tramite.Pago','U') IS NULL
            THEN 'NO EXISTE' ELSE 'EXISTE' END;
```

3. Creación controlada de tablas de laboratorio

Ejecutar únicamente en una base de datos de prácticas y con autorización. Antes de crear objetos, comprobar si ya existen tablas equivalentes bajo otro esquema. Este script no elimina objetos existentes.

```
IF NOT EXISTS (SELECT 1 FROM sys.schemas WHERE name='Tramite')
    EXEC('CREATE SCHEMA Tramite');
GO
IF OBJECT_ID('Tramite.Tramite','U') IS NULL
BEGIN
    CREATE TABLE Tramite.Tramite (
        IdTramite INT IDENTITY(1,1) PRIMARY KEY,
        IdEstadoTramite INT NOT NULL,
        FechaInicio DATE NOT NULL,
        IdMatricula INT NULL,
        IdTipoTramite INT NULL,
        IdCoordinacion INT NULL
    );
END;
GO
IF OBJECT_ID('Tramite.Pago','U') IS NULL
BEGIN
    CREATE TABLE Tramite.Pago (
        IdPago INT IDENTITY(1,1) PRIMARY KEY,
        IdTramite INT NOT NULL,
        FechaPago DATE NULL,
        Monto DECIMAL(10,2) NULL,
        NroRecibo VARCHAR(30) NULL,
        TipoPago VARCHAR(30) NULL
    );
END;
GO
```

4. Implementación de índices compuestos

```
IF NOT EXISTS (SELECT 1 FROM sys.indexes
               WHERE object_id=OBJECT_ID('Tramite.Tramite')
```

```
                AND name='IX_Tramite_Estado_Fecha')
    CREATE NONCLUSTERED INDEX IX_Tramite_Estado_Fecha
    ON Tramite.Tramite (IdEstadoTramite, FechaInicio)
    INCLUDE (IdMatricula, IdTipoTramite, IdCoordinacion);
GO
IF NOT EXISTS (SELECT 1 FROM sys.indexes
               WHERE object_id=OBJECT_ID('Tramite.Pago')
                 AND name='IX_Pago_Tramite_Fecha')
    CREATE NONCLUSTERED INDEX IX_Pago_Tramite_Fecha
    ON Tramite.Pago (IdTramite, FechaPago)
    INCLUDE (Monto, NroRecibo, TipoPago);
GO
```

5. Consulta de comprobación final

```
SELECT s.name AS Esquema, t.name AS Tabla,
       i.name AS NombreIndice, i.type_desc AS TipoIndice
FROM sys.indexes i
JOIN sys.tables t ON i.object_id=t.object_id
JOIN sys.schemas s ON t.schema_id=s.schema_id
WHERE s.name='Tramite'
  AND t.name IN ('Tramite','Pago')
  AND i.name IS NOT NULL
ORDER BY t.name,i.name;
```

Resultado esperado: deben visualizarse los índices IX_Tramite_Estado_Fecha e IX_Pago_Tramite_Fecha, ambos de tipo NONCLUSTERED, además de los índices de clave primaria. Se trata de una expectativa para una ejecución posterior, no de un resultado verificado.

6. Interpretación y recomendaciones

El índice de trámites utiliza IdEstadoTramite como columna de igualdad y FechaInicio como columna de rango; las columnas INCLUDE pueden cubrir consultas que las recuperen. El índice de pagos combina IdTramite y FechaPago. La existencia de índices no demuestra por sí sola mejoras de rendimiento: para evaluarlas se necesitan datos, consultas representativas y planes de ejecución.

Si la consulta vuelve a aparecer vacía, verificar DB_NAME(), los nombres de los objetos y los permisos. No crear tablas de laboratorio en una base de producción. Guardar una captura de la verificación final y diferenciarla de las capturas originales de la guía.

7. Conclusión del caso

La causa de los resultados vacíos fue la ausencia de las tablas Tramite.Tramite y Tramite.Pago en el entorno consultado. El procedimiento propuesto permite reproducir la práctica de forma controlada, crear los índices y comprobarlos mediante las vistas de catálogo de SQL Server.